

Especificación y Dinámica del Flujo de Trabajo en el Espacio de Trabajo Multidisciplinario

Prof. César Rodríguez
Departamento de Electrónica
Agente IA (Orquestador)

5 de julio de 2026

Índice

1. Introducción	2
2. Arquitectura de Tres Capas	2
3. Servidores MCP (Model Context Protocol)	2
3.1. Filosofía y Rol	3
3.2. Esquema de Activación	3
4. Dinámica del Flujo ante una Instrucción del Usuario	3
5. Protocolo ante la Ausencia de Infraestructura	4
6. Integración con Canales Externos (ej: Telegram)	4
6.1. Estructura de la Integración	5
6.2. Esquema de Conectividad (stdio vs SSE)	5
6.3. Flujo de Retorno	5

1. Introducción

El presente documento especifica la arquitectura, dinámica operacional y protocolos de ejecución que gobiernan este espacio de trabajo multidisciplinario de Electrónica, IoT, Diseño de Circuitos Integrados (EDA) y Redacción Académica.

Con el fin de mitigar la naturaleza probabilística y los sesgos de alucinación inherentes a los modelos de lenguaje de gran tamaño (LLM), el espacio de trabajo implementa una rigurosa **arquitectura de tres capas**. Esta estructura separa las intenciones declarativas de alto nivel de las acciones imperativas y deterministas, garantizando un índice de fiabilidad técnica óptimo y un comportamiento robusto ante errores.

2. Arquitectura de Tres Capas

La arquitectura del sistema se divide en tres niveles con responsabilidades claramente demarcadas:

1. Capa 1: Directivas (Directives - directives/)

Constituye el manual de operaciones de la organización. Consiste en procedimientos estandarizados de operación (SOPs) definidos exclusivamente en formato YAML. Cada archivo especifica de forma declarativa un flujo de trabajo repetible: el objetivo principal (*Goal*), los datos y variables de entrada (*Required inputs*), la secuencia lógica de pasos (*Steps*), las salidas esperadas (*Expected outputs*) y los protocolos de mitigación de fallas (*Edge cases*).

2. Capa 2: Orquestación (Orchestration - El Agente / flujo_*.py)

Actúa como el motor inteligente de toma de decisiones. Es el intermediario entre la intención conceptual del usuario y los recursos del sistema. El agente interpreta la directiva de la Capa 1, valida y sanitiza los parámetros de entrada, gestiona el flujo secuencial y el presupuesto de reintentos (*Retry Budget*), captura errores, y registra el progreso de forma persistente en `.tmp/run_state.json`.

3. Capa 3: Ejecución (Execution - execution/)

Consiste en scripts deterministas en Python o contenedores Docker aislados. Estos scripts tienen una única responsabilidad (ej. compilar LaTeX, vectorizar documentos, convertir PDFs a imágenes). Reciben sus parámetros a través de argumentos de interfaz de línea de comandos (CLI), no improvisan ni razonan, validan estrictamente sus salidas y retornan códigos de salida estándar (0 en caso de éxito, y códigos de error categorizados en caso de fallo).

3. Servidores MCP (Model Context Protocol)

Los servidores MCP (archivos con el prefijo `mcp_*_server.py`) son aplicaciones construidas sobre el framework **FastMCP** de Anthropic que exponen herramientas (*tools*) específicas para que clientes externos o interfaces de agente interactúen con el espacio de trabajo de manera estandarizada.

3.1. Filosofía y Rol

Los servidores MCP no contienen lógica de ejecución compleja. Su único propósito es servir de fachada adaptadora. Reciben una petición estructurada JSON-RPC, resuelven las rutas de archivos de manera absoluta, invocan el script orquestador correspondiente (Capa 2) utilizando el mismo intérprete de Python activo en el entorno virtual (`sys.executable`), leen el resultado de la ejecución en `.tmp/run_state.json`, y devuelven una respuesta Markdown limpia y amigable al cliente.

3.2. Esquema de Activación

El ciclo de vida del servidor MCP está diseñado para ser eficiente y no consumir recursos del sistema de forma innecesaria:

- **Activación Automática bajo Demanda (Client-spawned):** Es el método pre-determinado. Cuando el usuario abre un cliente de IA compatible (como Claude Desktop, Cursor o VS Code), este lee el archivo de configuración, lanza el proceso de Python en segundo plano y se comunica de manera bidireccional mediante el canal estándar de entrada y salida (`stdio`).
- **Activación para Pruebas Manuales:** Un desarrollador puede levantar el servidor localmente en terminal para depuración ejecutando de forma directa:

```
python3 mcp_latex_server.py
```

- **Desactivación Automática:** Cuando el cliente que invocó el servidor se cierra, el sistema operativo envía una señal de terminación al subproceso de Python, cerrando el servidor limpiamente y liberando la memoria.

4. Dinámica del Flujo ante una Instrucción del Usuario

Cuando el usuario ingresa una instrucción en este espacio de trabajo, se desencadena la siguiente secuencia lógica de ejecución:

Instrucción(Usuario) → Agente(Entendimiento) → ServidorMCP →
Orquestador(Capa2) → Scripts(Capa3) → Notificación(Física)

1. **Trigger (Detonador):** El usuario introduce una instrucción en lenguaje natural (ej: *"Evalúa el examen en PDF del alumno"*).
2. ****Análisis y Consulta Semántica:**** El agente IA (Capa 2) procesa la entrada, verifica su entorno y consulta la memoria semántica persistente (ChromaDB) en búsqueda de contextos previos o fallas conocidas similares.
3. ****Selección de Herramienta:**** El cliente de IA invoca la herramienta registrada en el servidor MCP relevante (ej. `evaluar_examen_estudiante`).
4. **Llamada al Orquestador (Capa 2):** El servidor MCP traduce la llamada JSON-RPC y ejecuta el script director (ej. `flujo_evaluar_examen.py`).

5. **Ejecución de Operarios (Capa 3):** El orquestador ejecuta de forma secuencial y determinista los scripts en `execution/`, guardando el estado incremental y los códigos de salida de cada paso en `.tmp/run_state.json`.
6. **Verificación y Auto-curación (Self-healing):** Si un operario falla (código de salida $\neq 0$), el orquestador analiza la salida del error, ajusta dinámicamente las entradas o el entorno, y realiza hasta 3 reintentos.
7. **Alerta y Respuesta Final:** El orquestador ejecuta `execution/alert_user.py` para generar una alerta auditiva exitosa (`success`) en la computadora anfitriona y devuelve el reporte final al cliente.

5. Protocolo ante la Ausencia de Infraestructura

Si el usuario solicita realizar una tarea completamente novedosa para la cual el espacio de trabajo no tiene ninguna directiva ni código preexistente, el agente actúa bajo el principio de **Mantenimiento y Evolución de Capacidades** siguiendo estos pasos de forma autónoma:

1. **Diseño de Solución:** Analiza los requerimientos, diagnostica el sistema en búsqueda de librerías faltantes y le presenta al usuario un plan estructurado bajo la arquitectura de 3 capas.
2. **Capa 1 (Manual):** Crea un archivo YAML en `directives/` detallando la nueva rutina lógica estándar.
3. **Capa 3 (Ejecutores):** Escribe scripts en `execution/` robustos, tipados, con control riguroso de excepciones y parámetros CLI estructurados.
4. **Capa 2 (Director):** Programa un script orquestador `flujo_*.py` que coordine a los ejecutores y maneje el estado de ejecución de forma persistente.
5. **MCP (Interfaz):** Desarrolla un nuevo adaptador MCP `mcp_*_server.py` utilizando `FastMCP` para exponer la nueva herramienta al exterior.
6. **Bucle de Prueba y Ajuste (Self-annealing):** Ejecuta de forma integrada el flujo completo, capturando y resolviendo cualquier fallo sintáctico o de lógica en el proceso de forma automática.

Al completar este flujo, no solo se resuelve la instrucción inmediata del usuario, sino que se deja el espacio de trabajo permanentemente expandido y preparado para automatizaciones futuras.

6. Integración con Canales Externos (ej: Telegram)

La infraestructura basada en la arquitectura de 3 capas y servidores MCP permite una integración nativa y sumamente robusta con canales externos como plataformas de mensajería (Telegram, Slack, Discord, etc.).

6.1. Estructura de la Integración

Un bot de Telegram desarrollado en Python (usando `python-telegram-bot`) puede actuar como un cliente MCP altamente eficaz:

1. El usuario envía un mensaje de texto o un archivo PDF desde su teléfono celular al chat del bot de Telegram.
2. El bot recibe la petición en la nube o en el servidor.
3. El bot actúa como cliente e invoca la herramienta MCP del espacio de trabajo.

6.2. Esquema de Conectividad (stdio vs SSE)

* **Canal Local (stdio):** Si el bot de Telegram se ejecuta directamente en la misma máquina o servidor local donde reside el espacio de trabajo de Electrónica, interactúa directamente con el servidor MCP a través del transporte de entrada/salida estándar (stdio) lanzando el proceso local de Python. * **Canal Remoto (SSE - Server-Sent Events):** Si el bot está alojado de forma remota o en la nube, se puede activar el servidor MCP en modo HTTP/SSE. Para proteger la máquina local, se establece un túnel seguro y cifrado (como **Cloudflare Tunnel** o `*ngrok*` con cifrado TLS/SSL de extremo a extremo). El bot de Telegram remoto realiza peticiones HTTP seguras a través de este túnel para invocar el pipeline de compilación de LaTeX o de análisis de imágenes de forma segura sin exponer puertos locales al internet público.

6.3. Flujo de Retorno

Una vez que el orquestador local de la Capa 2 finaliza su ejecución, el servidor MCP retorna el reporte JSON/Markdown y la ubicación del PDF compilado. El bot de Telegram lee este archivo local, lo sube de forma automatizada mediante el API de Telegram y lo deposita en tu chat, completando una interacción remota transparente y sumamente potente.